

UNIVERSIDAD DE



**ESTUDIO DE TÉCNICAS DE MONTE
CARLO Y MONTE CARLO TREE SEARCH
APLICADAS AL APRENDIZAJE POR
REFUERZO EN JUEGOS DE ESTRATEGIA**

Ana Gil Molina

09/07/2025

Índice

- 1. Introducción**
- 2. Algoritmos de toma de decisiones en juegos**
- 3. Metodología e implementación**
- 4. Experimentos y resultados**
- 5. Conclusiones**

Introducción

Introducción

Teoría de juegos

- ▶ La **teoría de juegos** es una disciplina que estudia situaciones estratégicas en las que los jugadores deben tomar decisiones considerando sus propias posibles acciones, pero también las de los demás jugadores.
- ▶ Especialmente relevante en diversas disciplinas, como la economía, las ciencias políticas, la biología o el derecho.
- ▶ Proporciona una base teórica fundamental para comprender y diseñar juegos de estrategia.

Introducción

Teoría de juegos

Teoría de la elección racional: *La acción escogida por un jugador es al menos tan buena, según sus preferencias, como cualquier otra acción disponible.*

Cualquier representación de un juego debe contener los siguientes elementos:

1. Una lista de jugadores.
2. Una descripción completa de las posibles acciones de los jugadores.
3. Una descripción de lo que los jugadores saben cuando actúan.
4. Una especificación de cómo las acciones de los jugadores llevan a los resultados.
5. Una especificación de las preferencias de los jugadores sobre los resultados.

Introducción

Forma normal de los juegos

- ▶ Sea un conjunto de jugadores $N = \{1, 2, \dots, n\}$. Sea un jugador $i \in N$.
- ▶ Se denota s_i una estrategia particular del jugador i , S_i el conjunto de todas las posibles estrategias del jugador i , $s = (s_1, \dots, s_n)$ un perfil de estrategias, y $s_{-i} = (s_1, \dots, s_{i-1}, s_{i+1}, \dots, s_n)$ un perfil de estrategias con las estrategias particulares de todos los jugadores excepto la del jugador i . Entonces $s = (s_i, s_{-i})$. Además, $S_{-i} = \prod_{j \neq i} S_j$, y $S = \prod_{i \in N} S_i$ es el conjunto de todos los posibles perfiles de estrategias.
- ▶ Sea la función de utilidad $u_i : S \rightarrow \mathbb{R}$ tal que $u_i(s) = u_i(s_1, \dots, s_n) = u_i(s_i, s_{-i})$ (en ocasiones se usa la función de beneficio $\pi_i : S \rightarrow \mathbb{R}$ tal que $\pi_i(s) = \pi_i(s_1, \dots, s_n) = \pi_i(s_i, s_{-i})$).

Entonces $G = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$ representa un **juego en forma normal**.

Introducción

Forma normal de los juegos

Un juego en forma normal con dos jugadores y espacio de estrategias finito se puede describir mediante matrices, por lo que en este caso estos juegos también se llaman **juegos matriciales**.

Introducción

Forma normal de los juegos

Example

Sea el juego $G = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$ dado por:

- ▶ Jugadores: $N = \{1, 2\}$
- ▶ Conjuntos de estrategias:
 - ▶ $S_1 = \{T, B\}$
 - ▶ $S_2 = \{L, M, R\}$
- ▶ Payoffs:
 - ▶ $u_1(T, L) = 4, u_1(T, M) = 2, u_1(T, R) = 0, u_1(B, L) = 5, u_1(B, M) = 5, u_1(B, R) = -4$
 - ▶ $u_2(T, L) = 3, u_2(T, M) = 7, u_2(T, R) = 4, u_2(B, L) = 5, u_2(B, M) = -1, u_2(B, R) = -2$

Introducción

Forma normal de los juegos

Example

Entonces el juego G se puede representar matricialmente como:

		Jugador 2		
		L	M	R
Jugador 1	T	4, 3	2, 7	0, 4
	B	5, 5	5, -1	-4, -2

Introducción

Forma extensiva de los juegos

- ▶ Existen juegos donde los movimientos se dan de forma secuencial, es decir, donde los jugadores mueven por turnos.
- ▶ En estos juegos secuenciales, los jugadores pueden observar los movimientos del resto de jugadores antes de elegir un movimiento.
- ▶ Se usa un diagrama de árbol, llamada **representación en forma extensiva** del juego, donde:
 - ▶ Los nodos marcan los momentos en los que los jugadores deben tomar una decisión.
 - ▶ Los arcos indican las distintas acciones que los jugadores pueden elegir a partir de cada nodo.

Introducción

Forma extensiva de los juegos

Example

- ▶ Hay un mercado dominado por un monopolista (M).
- ▶ Una nueva compañía entrante (E) decide si mantenerse fuera (F) o ingresar (I) al mercado del monopolista.
- ▶ Si la entrante ingresa, el monopolista puede acomodarse (A) o luchar (L).

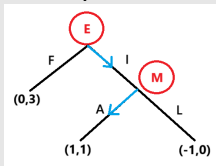


Figura 1: Diagrama de árbol en el que los nodos están conectados por arcos que representan las acciones tomadas por los jugadores.

Introducción

Teoría de juegos

- ▶ La teoría de juegos clásica proporciona una base matemática para analizar decisiones estratégicas entre varios jugadores. Sin embargo, su aplicabilidad práctica en entornos de gran complejidad es limitada.
- ▶ En los últimos años se han desarrollado nuevos algoritmos computacionales que permiten abordar problemas estratégicos mediante aproximaciones, útiles en juegos de tablero.

Algoritmos de toma de decisiones en juegos

Algoritmos de toma de decisiones en juegos

Monte Carlo

En el enfoque más simple, denominado **Monte Carlo plano (flat Monte Carlo)**:

- ▶ Las acciones disponibles desde un estado se eligen de forma uniforme.
- ▶ Para cada acción se realizan numerosas simulaciones hasta el final de la partida.

Sea $Q(s, a)$ la recompensa esperada de tomar la acción a en el estado s . Entonces:

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s)} \mathbb{I}_i(s, a) z_i \quad (1)$$

Algoritmos de toma de decisiones en juegos

Monte Carlo

donde:

- ▶ $N(s, a)$ es el número de veces que la acción a ha sido seleccionada desde el estado s .
- ▶ $N(s)$ es el número de simulaciones que han pasado por s .
- ▶ z_i es el resultado de la i -ésima simulación jugada desde s .
- ▶ $\mathbb{I}_i(s, a)$ es una función que vale 1 si en la simulación i se tomó la acción a desde s , y 0 en caso contrario.

Se selecciona la acción con mayor recompensa esperada $Q(s, a)$.

Algoritmos de toma de decisiones en juegos

Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) combina la idea de la evaluación mediante simulaciones con una estructura de árbol y una política de selección que centra los recursos computacionales en las partes importantes del árbol. Esta política equilibra:

- ▶ **Exploración** de los estados que han tenido pocas jugadas, permitiendo mejorar las estimaciones de dichos estados.
- ▶ **Explotación** de los estados que han dado buenos resultados en jugadas anteriores.

Algoritmos de toma de decisiones en juegos

Monte Carlo Tree Search: Pasos

1. **Selección:** Comenzando en el nodo raíz, se elige un movimiento según la política de selección, alcanzando un nodo hijo, y se repite el proceso sucesivamente, recorriendo el árbol hasta alcanzar un nodo hoja, esto es, un nodo que tiene al menos un hijo potencial al que no se le ha aplicado ninguna simulación todavía.
2. **Expansión:** A partir del nodo hoja seleccionado, crear un nodo hijo, es decir, un movimiento válido desde el nodo hoja, haciendo crecer el árbol.
3. **Simulación:** Se completa una jugada desde el nodo hijo generado, eligiendo movimientos para ambos jugadores de acuerdo a una política. Estos movimientos no se almacenan en el árbol.
4. **Retropropagación:** Se usan los resultados de la simulación para actualizar información en los nodos, recorriéndolos desde el nodo expandido hasta el nodo raíz.

Algoritmos de toma de decisiones en juegos

MCTS: Política de selección ϵ -greedy

El método ϵ -**greedy** consiste en:

- ▶ Con probabilidad $1 - \epsilon$ se selecciona la acción **greedy**, es decir, la acción con mayor recompensa estimada (explotación).
- ▶ Con probabilidad ϵ se selecciona una acción aleatoria (exploración).

Algoritmos de toma de decisiones en juegos

MCTS: Política de selección UCB

El método de **límites superiores de confianza** o **Upper Confidence Bound (UCB)** mide el potencial de cada acción con un índice que está formado por la suma de dos términos:

$$ucb(a) = Q(a) + u(a) \quad (2)$$

donde:

- ▶ $Q(a)$ es un término de explotación.
- ▶ $u(a)$ es un término de exploración (límite superior).

Algoritmos de toma de decisiones en juegos

MCTS: Política de selección UCB

Este método siempre selecciona la acción más codiciosa para maximizar el límite superior de confianza:

$$a^* = \arg \max_{a \in A} ucb(a) \quad (3)$$

Algoritmos de toma de decisiones en juegos

MCTS: Política de selección UCB

Existen varias versiones del algoritmo UCB, entre las cuales destacan las dos siguientes:

- ▶ **UCB1:** cuyo valor viene dado por:

$$ucb1(a) = Q(a) + c \cdot \sqrt{\frac{\ln t}{N(a)}} \quad (4)$$

donde:

- ▶ t es el número total de simulaciones hasta ahora.
- ▶ $N(a)$ es el número de veces que la acción a ha sido seleccionada hasta el momento.
- ▶ c es un parámetro de ajuste que controla el grado de exploración.

Algoritmos de toma de decisiones en juegos

MCTS: Política de selección UCB

- ▶ **UCB2:** cuyo valor viene dado por:

$$ucb2(a) = Q(a) + \sqrt{\frac{(1 + \alpha) \ln\left(\frac{e \cdot t}{\tau(k_a)}\right)}{2\tau(k_a)}} \quad (5)$$

donde:

- ▶ t es el número total de simulaciones hasta ahora.
- ▶ $0 < \alpha < 1$ es un parámetro de ajuste para el balance entre exploración y explotación.
- ▶ k_a es el número de épocas de la acción a .
- ▶ $\tau(k_a) = \lceil (1 + \alpha)^{k_a} \rceil$.

Selección por **épocas**: cuando una acción a es seleccionada, esta se ejecuta por $\lceil \tau(k_a + 1) - \tau(k_a) \rceil$ simulaciones. Al terminar, se vuelve a evaluar cuál es la siguiente acción a seleccionar.

Algoritmos de toma de decisiones en juegos

MCTS: Política de selección Softmax

Con el método **Softmax** las acciones son seleccionadas de acuerdo a la distribución de probabilidad dada por la función de Gibbs (softmax):

$$\pi_t(a) = P(A_t = a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^k e^{Q_t(b)/\tau}} \quad (6)$$

Además, existe una versión adaptativa del método Softmax que ajusta el parámetro τ dinámicamente en cada iteración.

Metodología e implementación

Metodología e implementación

Todos los algoritmos, juegos y herramientas han sido implementados en Python. Las principales librerías de Python utilizadas son:

- ▶ `sys`: para modificar la variable de entorno `sys.path`, permitiendo agregar directorios al `path`.
- ▶ `os`: interacción con el sistema operativo, manejo de archivos, rutas y procesos.
- ▶ `pandas` (versión 2.0.3): para manipulación y análisis de tablas.
- ▶ `random`: generar números aleatorios.
- ▶ `numpy` (versión 1.23.5): para cálculo numérico y manejo eficiente de arrays multidimensionales.
- ▶ `copy`: permite realizar copias superficiales y profundas de objetos en Python.
- ▶ `graphviz` (versión 0.20.3): interfaz en Python para la creación y visualización de grafos mediante el motor de Graphviz.
- ▶ `abc`: soporte para clases abstractas.

Metodología e implementación

La implementación se ha organizado mediante diferentes directorios conteniendo el código necesario para los experimentos:

- ▶ `games/`: Contiene las implementaciones de los juegos, sobre los que se prueban los algoritmos. Incluye los archivos:
 - ▶ `nim.py`: Implementación del juego Nim.
 - ▶ `connect4.py`: Implementación del juego 4 en raya.
 - ▶ `othello.py`: Implementación del juego Othello.
 - ▶ `go.py`: Implementación del juego Go.
- ▶ `methods/`: Contiene los métodos implementados para la toma de decisiones. Incluye los archivos:
 - ▶ `monte_carlo.py`: Implementación del método de Monte Carlo.
 - ▶ `monte_carlo_tree_search.py`: Implementación del método de Monte Carlo Tree Search.

Metodología e implementación

- ▶ `selection/`: Contiene las diferentes políticas de selección explicadas, que se utilizan dentro del algoritmo MCTS. Incluye:
 - ▶ `algorithm.py`: Clase genérica para los algoritmos de selección.
 - ▶ `epsilon_greedy.py`: Implementación de la política de selección ϵ -greedy.
 - ▶ `ucb1.py`: Implementación de la política de selección UCB1.
 - ▶ `ucb2.py`: Implementación de la política de selección UCB2.
 - ▶ `softmax.py`: Implementación de la política de selección Softmax.
 - ▶ `adaptive_softmax.py`: Implementación de la política de selección Softmax adaptativo.
- ▶ `graphviz/`: Se ha desarrollado una herramienta de depuración para visualizar gráficamente el árbol de búsqueda generado por MCTS, mediante la biblioteca `graphviz`. Esta herramienta se encuentra en el archivo:
 - ▶ `MCTS_graphviz.py`: Implementación de la herramienta de depuración para MCTS.

Metodología e implementación

- ▶ `notebooks`: Se han desarrollado notebooks de Jupyter en los que se realizan simulaciones, visualizando los resultados y comparando el rendimiento de los distintos métodos en cada juego. Se encuentra un notebook por cada juego:
 - ▶ `nim_simulations.ipynb`: Notebook con los resultados para el juego Nim.
 - ▶ `connect4_simulations.ipynb`: Notebook con los resultados para el juego 4 en raya.
 - ▶ `othello_simulations.ipynb`: Notebook con los resultados para el juego Othello.
 - ▶ `go_simulations.ipynb`: Notebook con los resultados para el juego Go.

Metodología e implementación

Todos los experimentos y cálculos se realizaron en un entorno local, utilizando un equipo portátil con las siguientes características:

- ▶ **Fabricante del dispositivo:** HP
- ▶ **Sistema operativo:** Windows 11 Home, versión 24H2 (build 26100.4484)
- ▶ **Procesador:** Intel Core i3-8130U CPU @ 2.20GHz (2 núcleos, 4 hilos)
- ▶ **Memoria RAM:** 8 GB
- ▶ **Arquitectura del sistema:** 64 bits (x64)
- ▶ **Entorno de ejecución:** Python 3.8.5 ejecutado localmente
- ▶ **Entorno de desarrollo:** Jupyter Notebook y ejecución directa desde scripts locales

No se ha hecho uso de GPU ni de paralelización explícita en los experimentos. Todos los cálculos se llevaron a cabo en un único núcleo del procesador en ejecución secuencial.

Experimentos y resultados

Experimentos y resultados

Los experimentos se han llevado a cabo utilizando cuatro juegos diferentes, de complejidad creciente: Nim (más sencillo), 4 en raya, Othello y Go (más complejo, con un espacio de búsqueda mucho más amplio). Para cada juego, se han llevado a cabo enfrentamientos entre pares de agentes:

- ▶ Nim: 1000 simulaciones por agente, 100 partidas por par.
- ▶ 4 en raya: 300 simulaciones por agente, 100 partidas por par.
- ▶ Othello: 300 simulaciones por agente, 100 partidas por par.
- ▶ Go: 100 simulaciones por agente, 10 partidas por par.

Experimentos y resultados

Nim

Nim de una pila es un juego estratégico en el que dos jugadores se turnan para retirar palos de un montón hasta que no quede ninguno.

Definition

Sean $a_1 < a_2 < \dots < a_k \in \mathbb{N}$ y $n \in \mathbb{N}$. Entonces $NIM(a_1, \dots, a_k; n)$ es el siguiente juego:

- ▶ Se comienza con un número n de palos y dos jugadores, Jugador 1 y Jugador 2.
- ▶ Los jugadores se alternan, siendo el Jugador 1 quien hace el primer movimiento.
- ▶ En cada turno, el jugador correspondiente retira $a \in \{a_1, a_2, \dots, a_k\}$ palos.
- ▶ Gana quien tome el último palo.

Experimentos y resultados

Nim

En particular, en este trabajo se usa una versión en la que los números $a_1 < a_2 < \dots < a_k \in \mathbb{N}$ son consecutivos y $a_1 = 1$. Para esta versión, $NIM(1, 2, \dots, k; n)$, con $k, n \in \mathbb{N}$, existe una estrategia ganadora:

- ▶ Las posiciones perdedoras son todas aquellas donde $N \equiv 0 \pmod{k+1}$.
- ▶ El objetivo es llevar al oponente a una de esas posiciones.
- ▶ Si empiezas en una posición perdedora, el oponente puede mantener el control con la misma técnica.

Agente A	Agente B	Ganó A	Ganó B	Runtimerow
MC	MCTS ϵ -greedy	44	56	3m 23,8s
MC	MCTS UCB1	32	68	3m 46,9s
MC	MCTS UCB2	84	16	4m 3,2s
MC	MCTS Softmax	71	29	4m 21,2s
MC	MCTS Softmax Adaptativo	66	34	4m 10,6s
MCTS ϵ -greedy	MCTS UCB1	27	73	56,6s
MCTS ϵ -greedy	MCTS UCB2	89	11	1m 23,5s
MCTS ϵ -greedy	MCTS Softmax	65	35	1m 36,9s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	66	34	1m 58,0s
MCTS UCB1	MCTS UCB2	89	11	1m 27,2s
MCTS UCB1	MCTS Softmax	83	17	1m 40,4s
MCTS UCB1	MCTS Softmax Adaptativo	87	13	2m 0,9s
MCTS UCB2	MCTS Softmax	30	70	2m 11,6s
MCTS UCB2	MCTS Softmax Adaptativo	17	83	2m 47,3s
MCTS Softmax	MCTS Softmax Adaptativo	50	50	2m 35,0s

Cuadro 1: Enfrentamientos por pares entre agentes en el juego Nim (100 partidas por par).

Experimentos y resultados

4 en raya

4 en raya es un juego de estrategia para dos jugadores que consiste en alinear cuatro fichas del mismo color en un tablero vertical. Sus reglas consisten en:

- ▶ El juego clásico se juega en un tablero de 7 columnas por 6 filas (aunque en esta implementación pueden tomar valores entre 4 y 10).
- ▶ Los jugadores se turnan para dejar caer una ficha en una de las columnas.
- ▶ La ficha cae hasta ocupar la posición más baja disponible en esa columna.
- ▶ El objetivo es alinear cuatro fichas consecutivas del mismo color (en línea horizontal, vertical o diagonal).
- ▶ Gana el primer jugador que consiga alinear cuatro fichas.

Agente A	Agente B	Ganó A	Ganó B	Empates	Runtime
MC	MCTS ϵ -greedy	57	41	2	14m 33,8s
MC	MCTS UCB1	52	47	1	14m 16,4s
MC	MCTS UCB2	96	4	0	10m 35,4s
MC	MCTS Softmax	72	25	3	13m 21,0s
MC	MCTS Softmax Adaptativo	76	21	3	13m 51,8s
MCTS ϵ -greedy	MCTS UCB1	39	56	5	5m 19,8s
MCTS ϵ -greedy	MCTS UCB2	92	7	1	4m 45,2s
MCTS ϵ -greedy	MCTS Softmax	68	30	2	5m 12,6s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	70	26	4	5m 33,5s
MCTS UCB1	MCTS UCB2	96	3	1	4m 20,9s
MCTS UCB1	MCTS Softmax	74	19	7	5m 25,5s
MCTS UCB1	MCTS Softmax Adaptativo	75	16	9	5m 31,9s
MCTS UCB2	MCTS Softmax	26	72	2	5m 18,5s
MCTS UCB2	MCTS Softmax Adaptativo	22	76	2	5m 41,8s
MCTS Softmax	MCTS Softmax Adaptativo	53	40	7	6m 16,4s

Cuadro 2: Enfrentamientos por pares entre agentes en el juego 4 en raya (100 partidas por par).

Experimentos y resultados

Othello

Othello es un juego de estrategia para dos jugadores en el que se colocan fichas en un tablero con el objetivo de tener la mayoría al final de la partida. Sus reglas son:

- ▶ El juego clásico se juega en un tablero de 8×8 casillas (aunque en esta implementación pueden tomar valores entre 4 y 10).
- ▶ Cada ficha tiene una cara blanca y una negra (en esta implementación se representan con 1 y 2).
- ▶ Comienza el jugador que lleva las fichas negras.
- ▶ Los jugadores se turnan para colocar una ficha con su color hacia arriba.
- ▶ Una jugada válida debe encerrar una o más fichas del oponente entre la nueva ficha y otra del mismo color.

Experimentos y resultados

Othello

- ▶ Las fichas del oponente que quedan encerradas se voltean al color del jugador que hizo la jugada.
- ▶ El juego termina cuando ningún jugador puede hacer una jugada válida.
- ▶ Gana quien tenga más fichas de su color en el tablero al final de la partida.

Agente A	Agente B	Ganó A	Ganó B	Empates	Runtime
MC	MCTS ϵ -greedy	1	99	0	10m 19,3s
MC	MCTS UCB1	0	100	0	9m 36,3s
MC	MCTS UCB2	23	77	0	9m 31,6s
MC	MCTS Softmax	0	100	0	9m 46,2s
MC	MCTS Softmax Adaptativo	0	100	0	10m 22,3s
MCTS ϵ -greedy	MCTS UCB1	0	100	0	3m 25,5s
MCTS ϵ -greedy	MCTS UCB2	46	54	0	3m 23,3s
MCTS ϵ -greedy	MCTS Softmax	24	76	0	3m 56,6s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	28	72	0	4m 20,3s
MCTS UCB1	MCTS UCB2	78	22	0	3m 32,4s
MCTS UCB1	MCTS Softmax	77	23	0	4m 5,3s
MCTS UCB1	MCTS Softmax Adaptativo	73	27	0	4m 21,9s
MCTS UCB2	MCTS Softmax	3	97	0	4m 4,2s
MCTS UCB2	MCTS Softmax Adaptativo	3	97	0	4m 50,5s
MCTS Softmax	MCTS Softmax Adaptativo	15	85	0	5m 9,5s

Cuadro 3: Enfrentamientos por pares entre agentes en el juego Othello (100 partidas por par).

Experimentos y resultados

Go

Go es un juego de estrategia para dos jugadores en el que se colocan fichas en un tablero con el objetivo de controlar el mayor territorio al final de la partida. Sus reglas son:

- ▶ El juego se juega en un tablero cuadrado con un número impar de filas y columnas entre 5 y 19 (por defecto 9×9).
- ▶ Los jugadores se turnan para colocar fichas de su color en las intersecciones vacías del tablero.
- ▶ Una jugada válida debe respetar las reglas de no suicidio y Superko situacional:
 - ▶ Regla de no suicidio: Un jugador no puede colocar una ficha que haga que su propio grupo quede sin libertades inmediatamente, a menos que esa jugada capture fichas enemigas que devuelvan libertades al grupo.
 - ▶ Regla de Superko situacional: Un jugador no puede realizar una jugada que haga que el tablero y el turno actual coincidan exactamente con una situación previa en la misma partida (para evitar ciclos infinitos).

Experimentos y resultados

Go

- ▶ Se llaman libertades a las casillas vacías adyacentes a una cierta ficha o grupo de fichas del mismo color.
- ▶ Si al colocar una ficha se rodean grupos enemigos sin libertades, estas fichas se capturan y se retiran del tablero.
- ▶ Un jugador puede pasar su turno, y si ambos jugadores pasan consecutivamente, el juego termina.
- ▶ El puntaje final de cada jugador se calcula sumando:
 - ▶ Las fichas en el tablero de dicho jugador.
 - ▶ Las fichas capturadas al oponente.
 - ▶ El territorio rodeado exclusivamente por fichas del color de dicho jugador.
 - ▶ La compensación Komi de 6,5 puntos para el jugador que no comenzó.
- ▶ Gana el jugador con más puntos al final de la partida.

Agente A	Agente B	Ganó A	Ganó B	Empates	Runtime
MC	MCTS ϵ -greedy	10	0	0	87m 34,6s
MC	MCTS UCB1	10	0	0	115m 9,0s
MC	MCTS UCB2	10	0	0	99m 25,4s
MC	MCTS Softmax	10	0	0	101m 23,2s
MC	MCTS Softmax Adaptativo	10	0	0	98m 4,3s
MCTS ϵ -greedy	MCTS UCB1	8	2	0	13m 25,5s
MCTS ϵ -greedy	MCTS UCB2	7	3	0	13m 37,2s
MCTS ϵ -greedy	MCTS Softmax	7	3	0	14m 59,7s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	6	4	0	14m 21,5s
MCTS UCB1	MCTS UCB2	7	3	0	12m 47,7s
MCTS UCB1	MCTS Softmax	7	3	0	14m 47,3s
MCTS UCB1	MCTS Softmax Adaptativo	8	2	0	12m 17,9s
MCTS UCB2	MCTS Softmax	8	2	0	13m 30,0s
MCTS UCB2	MCTS Softmax Adaptativo	7	3	0	15m 5,1s
MCTS Softmax	MCTS Softmax Adaptativo	5	5	0	12m 16,8s

Cuadro 4: Enfrentamientos por pares entre agentes en el juego Go (10 partidas por par).

Conclusiones

The slide features a white background with two large, overlapping geometric shapes at the bottom. On the left is a teal-colored triangle pointing upwards and to the right. On the right is a dark blue triangle pointing upwards and to the left. These two triangles meet at a point in the center of the bottom edge, creating a symmetrical, V-shaped negative space.

Conclusiones

- ▶ En juegos simples como Nim o 4 en raya, Monte Carlo básico muestra un buen funcionamiento frente a MCTS.
- ▶ Dependiendo del método de selección ha habido algunas diferencias en los resultados, por ejemplo, en Nim, con ϵ -greedy y con UCB1, el agente de MCTS vence a Monte Carlo básico, pero con otras políticas de selección, Monte Carlo muestra un mejor desempeño que MCTS.
- ▶ En 4 en raya, el agente de Monte Carlo ha logrado superar a MCTS en todos los escenarios.

Conclusiones

- ▶ En juegos más complejos como Othello, el rendimiento del agente de Monte Carlo básico se ha deteriorado considerablemente. En todos los casos, los agentes de MCTS han superado con creces a Monte Carlo, ganando cerca del 100 % de las partidas en la mayoría de enfrentamientos.
- ▶ En el juego Go, todavía de mayor complejidad, las limitaciones de Monte Carlo básico se han vuelto todavía más evidentes. Debido a la muy elevada complejidad computacional del juego, se ha debido tomar:
 - ▶ Un tablero lo más pequeño posible (5×5).
 - ▶ 100 simulaciones por agente.
 - ▶ 10 partidas por cada par de agentes enfrentados.

Conclusiones

- ▶ A pesar de que teóricamente el algoritmo MCTS debería superar al método de Monte Carlo básico en juegos complejos como Go, los resultados obtenidos en este trabajo muestran justo lo contrario. Este comportamiento puede explicarse por:
 1. **Número bajo de simulaciones:** solo 100 simulaciones por movimiento, lo cual es insuficiente para MCTS, ya que necesita realizar múltiples simulaciones para construir un árbol de búsqueda para identificar los nodos más prometedores. Monte Carlo realiza partidas aleatorias, lo cual puede explicar que su rendimiento no se degrade tanto.
 2. **Go con tablero muy reducido:** al usar un tablero de solo 5×5 , se simplifica mucho el espacio de juego. Monte Carlo puede generar jugadas razonables por pura casualidad.
 3. **Parámetros de MCTS no óptimos:** los parámetros seleccionados por defecto para las políticas de selección de MCTS pueden no estar bien ajustados para este juego.

Conclusiones

- ▶ En Go, el tiempo de ejecución ha sido:
 - ▶ En torno a 90 y 110 minutos en los pares en los que se enfrentaba Monte Carlo frente a MCTS.
 - ▶ Algo inferior a 15 minutos en los pares en los que se enfrentaban dos agentes de MCTS con distinta política de selección.
- ▶ Go es famoso por tener una combinatoria explosiva.
- ▶ Un programa de IA desarrollado por DeepMind llamado *AlphaGo*, que combina redes neuronales profundas, búsqueda mediante Monte Carlo Tree Search y aprendizaje por refuerzo para jugar a Go competitivamente en tableros de tamaño profesional (19×19 celdas), ganó en el año 2016, a uno de los mejores jugadores del mundo de Go.

Referencias I

-  Auer, Peter, Nicolò Cesa-Bianchi y Paul Fischer (2002). “Finite-time Analysis of the Multiarmed Bandit Problem”. En: *Machine Learning* 47.2–3, págs. 235-256. DOI: 10.1023/A:1013689704352. URL: <https://doi.org/10.1023/A:1013689704352>.
-  Browne, Cameron et al. (mar. de 2012). “A Survey of Monte Carlo Tree Search Methods”. En: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1.
-  Gasarch, William y John Purlito (2015). *NIM with Cash*.
-  MSMK The University of Science & Technology (ago. de 2024). *AlphaGo*. URL: <https://msmk.university/alphago/>.

Referencias II

-  Osborne, Martin J. (2004). *An Introduction to Game Theory*. New Delhi, India: Oxford University Press.
-  Russell, Stuart J. y Peter Norvig (2022). *Artificial Intelligence: A Modern Approach*. Fourth, Global. Harlow, United Kingdom: Pearson Education Limited.
-  Sutton, Richard S. y Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. Second. Cambridge, Massachusetts; London, England: MIT Press. ISBN: 978-0-262-19398-6.
-  Watson, Joel (2013). *Strategy: An Introduction to Game Theory*. Third. New York, NY: W. W. Norton Company. ISBN: 978-0393918380.

